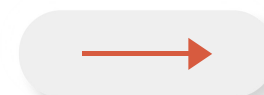
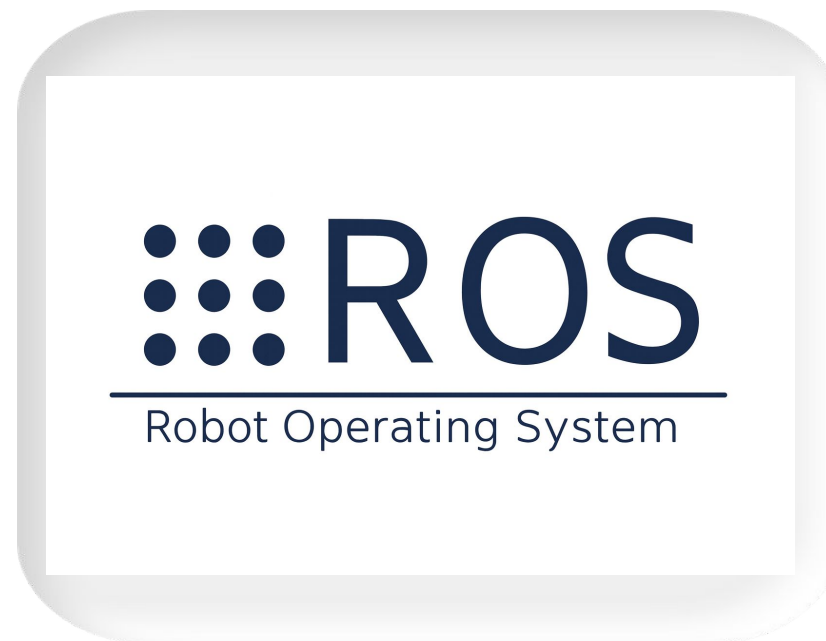




Nodelets



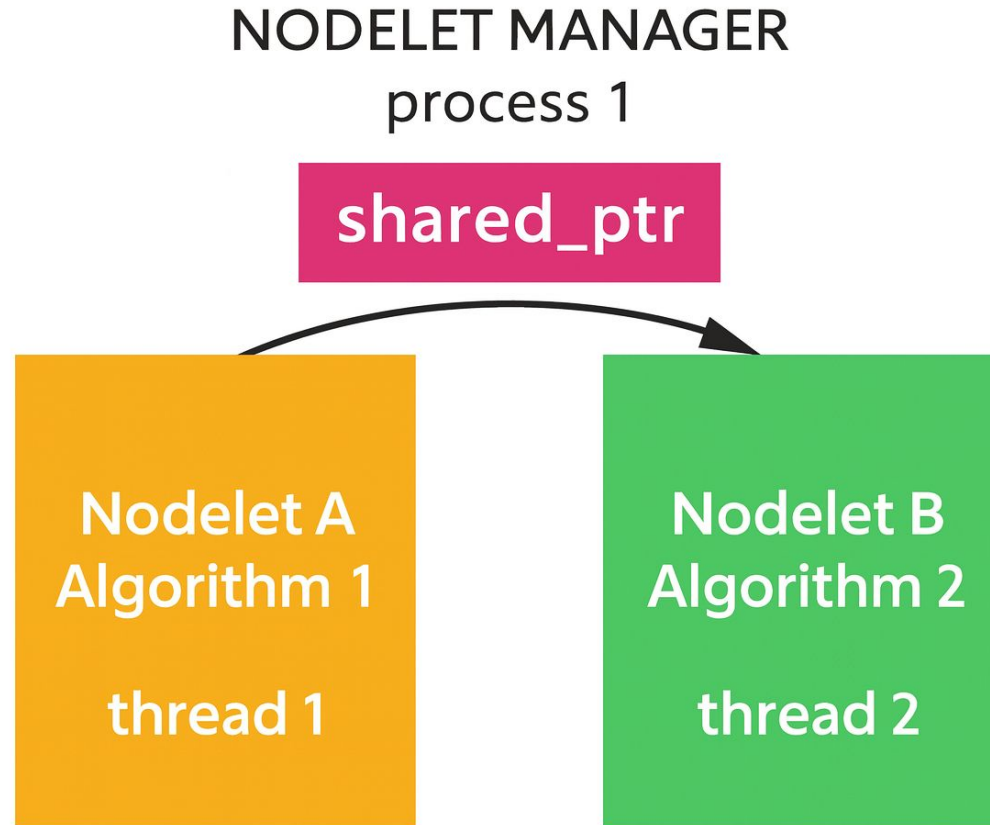


Lecture 09: Nodelets

What is a ROS Nodelet?

- A ROS Nodelet is a lightweight ROS component that lets multiple algorithms run inside the same process, allowing them to share data without copying.
- This makes communication much faster, especially for large sensor data like images or point clouds.

What is a ROS Nodelet?



What is a ROS Nodelet?

- It allows to
 - Run multiple algorithms inside one process
 - Use zero-copy transport → much faster data sharing
 - Good for camera, LiDAR, and image processing
 - Reduce CPU and memory usage
 - Use a nodelet base class to create your own nodelet
 - Use a NodeletLoader to load nodelets into a process
 - Often used with nodelet manager to organize nodelets

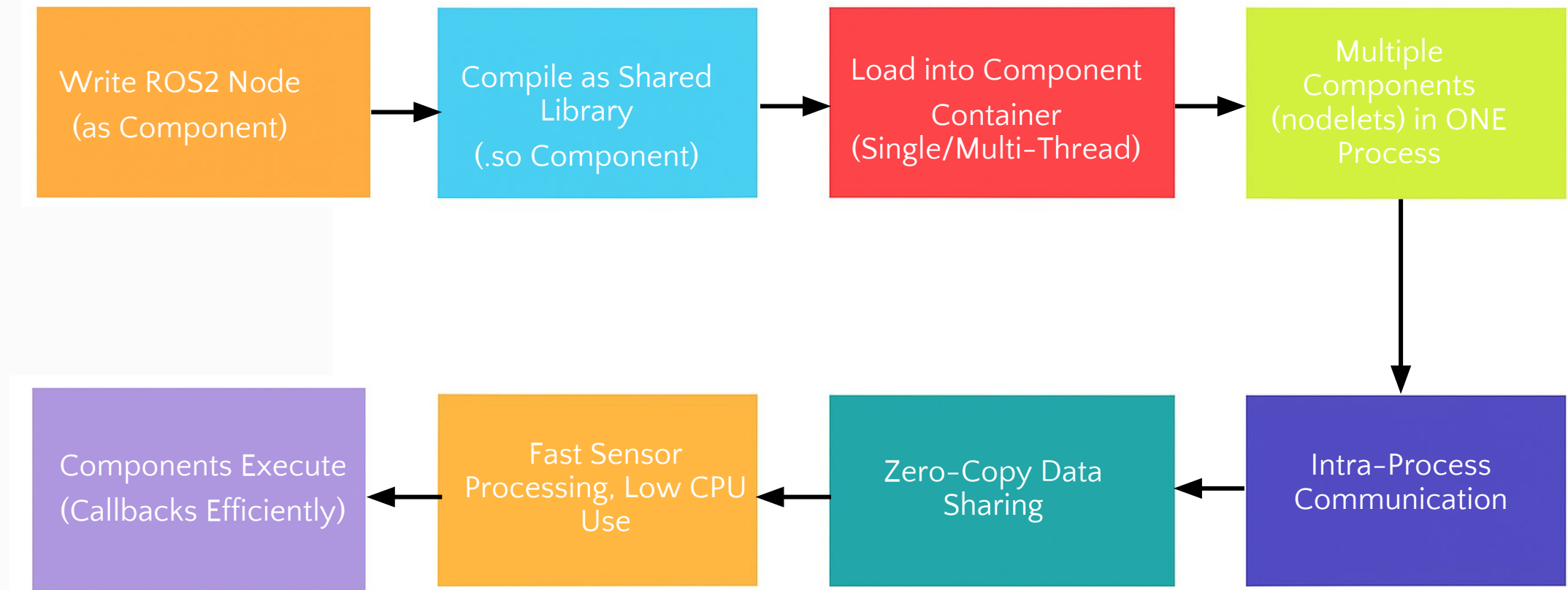
Question

- Why Nodelets Are Faster?

Question

- How do Nodelets work in ROS2? Draw the flowchart and briefly explain each block in the working flow.

How **Nodelets** Work in ROS2?



1. Write a ROS2 Node (as a Component)

- A component is just a normal ROS2 node written in a way that ROS2 can load it into a shared process.
 - You create a ROS2 node just like any normal node.
 - But instead of making an executable, you write it as a component class.
 - This allows ROS2 to load it dynamically later.

2. Compile as a Shared Library (.so)

- Instead of making a whole program, we make a library that ROS2 can plug in wherever it needs.
 - Components are not compiled into a single program.
 - They are turned into shared libraries (.so files).
 - This lets ROS2 load and unload them at runtime.

3. Load Component into a Component Container

- . The component container is the program that actually runs all the components together.
 - . A Component Container is like a “manager program” in ROS2.
 - . It loads one or many components.
 - . It can run single-threaded or multi-threaded.

4. Several Components (Nodelets) Run Inside the SAME Process

- Instead of each node being its own program, components share one program and one memory.
 - All components loaded into the container share one memory space.
 - They run together like apps inside one big program.
 - No need for separate executables.

5. Intra-Process Communication Enabled

- ROS2 automatically shares messages without copying them. This makes everything faster.
 - ROS2 detects that publishers and subscribers are in the same process.
 - It passes messages directly by pointer.
 - No serialization, no copying.

6. Zero-Copy Data Sharing

- Instead of copying big data from one node to another, ROS2 lets components use the same data in memory.
 - Large data (images, pointclouds, etc.) is NOT duplicated.
 - Only a pointer to the message is passed.
 - Great for performance.

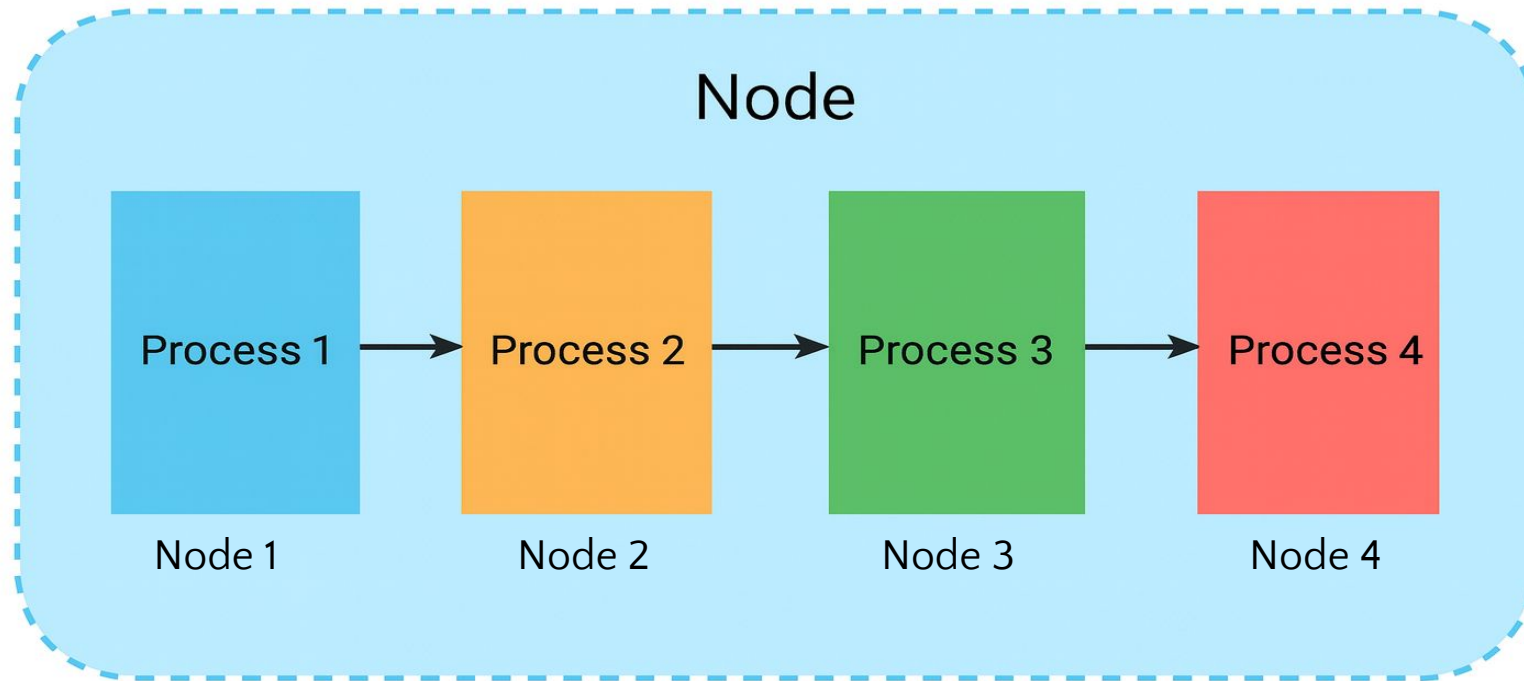
7. Fast Sensor Processing, Low CPU Use

- Sharing data directly makes the robot react faster and uses less CPU power.
 - No message copying
 - No serialization
 - Lower latency
 - Better real-time safety.

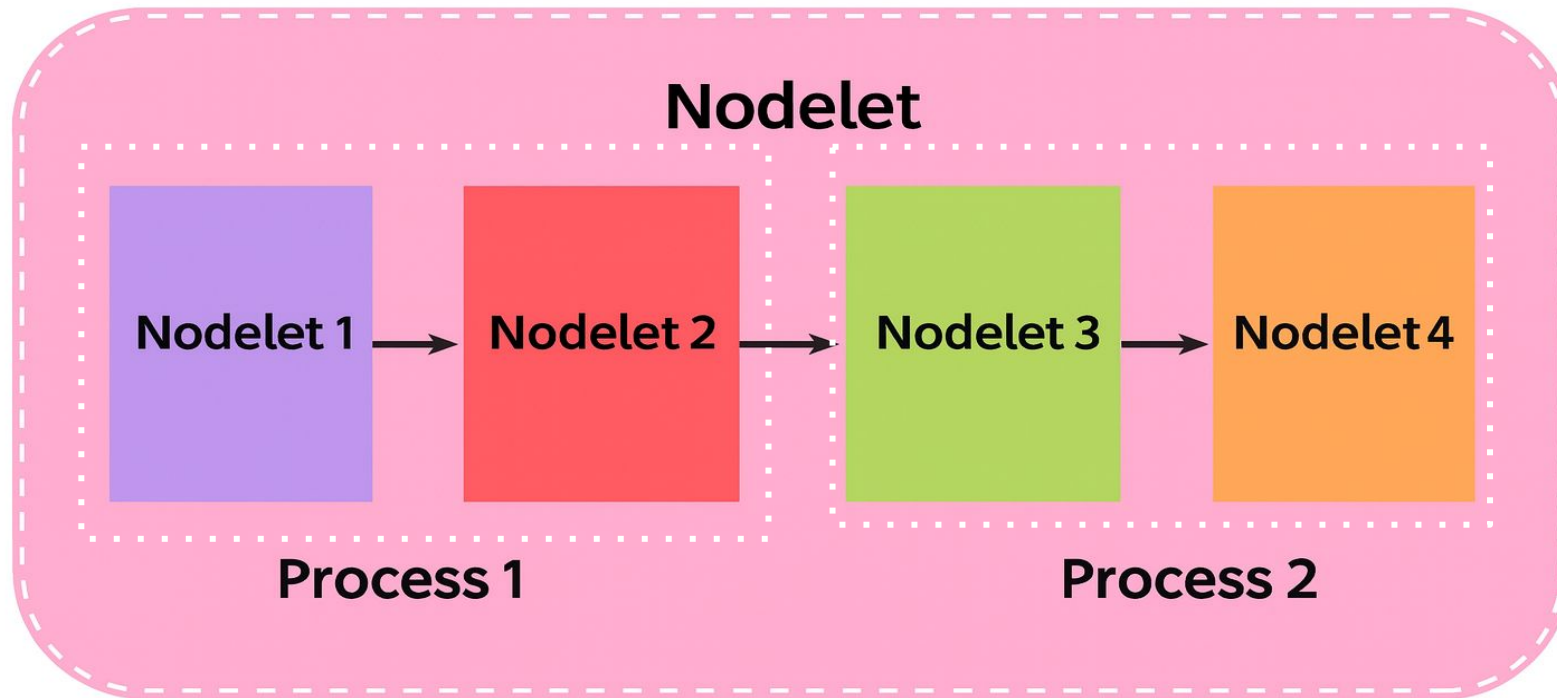
8. Components Execute Callbacks Efficiently

- ROS2 runs all the component algorithms smoothly inside one program, without wasting time on extra data transfers.
 - Each component has its own callback functions.
 - The container's executor runs them efficiently.
 - Execution can be multi-threaded for speed.

ROS Node vs Nodelet



ROS Node vs Nodelet



ROS **Node** vs **Nodelet**

Normal Node	Nodelet
Runs as its own process	Runs inside a shared process with other nodelets
Data is copied between nodes	Zero-copy data sharing (much faster)
Easier to debug and isolate	Faster but less isolated (a crash affects all nodelets)
Good for general ROS applications	Best for high-bandwidth sensors (camera, LiDAR, images)
Uses more CPU/memory due to copying	Saves CPU/memory by sharing data
Simple to start/stop	Needs a nodelet manager

Q & A





Thank
you!